Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#_07__)

Implementation of Arithmetic Logic Unit (ALU)

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B___

Date performed: __7TH October _____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:

In this task, we designed a 1-bit ALU in Verilog using gate-level modeling. We use the code of half adder and full adder, and also AND, OR and then we build 4x1 MUX where 4 inputs two selection line to decide which operation to perform then we call these all module in module of 1bit alu . The inputs A and B go through an XOR gate controlled by b_inv, so the same adder can do both addition and subtraction (A + B' + 1), and the testbench applies all input combinations to check the Result and Carry_out in the simulation.

Code:

```
//halfadder

module halfadder(a,b,s,c);

input a;

input b;

output s;

output c;

xor x1(s,a,b);

and a1(c,a,b);

endmodule



//fulladder

module fulladder(a,b,cin,s,c);

input a;

input b;

input cin;

output s;

output c;
```

```
wire t1;

wire t2;

wire t3;

halfadder U1(a,b,t1,t2);

halfadder U2(t1,cin,s,t3);

or o1(c,t2,t3);

endmodule



//and

module logic_and(a,b,c);

input a;

input b;

output c;

and a1(c,a,b);

endmodule



//or

module logic_or(a,b,c);

input a;

input b;

output c;

or a1(c,a,b);

endmodule
```

```
//mux

module mux4x1(d0,d1,d2,d3,s1,s0,y);

input d0;//AND 00

input d1;//OR  01

input d2;//10 ADD/SUB

input d3;//11 NOT IN USE

input s1;

input s0;

output y;

wire t1;

wire t0;

wire b0;

wire b1;

wire b2;

wire b3;

not n2(t1,s1);

not n3(t0,s0);

and a0(b0,d0,t1,t0);

and a1(b1,d1,t1,s0);

and a2(b2,d2,s1,t0);

and a3(b3,d3,s1,s0);

or o1(y,b0,b1,b2,b3);
```

```
endmodule



//1_bitALU

module alubit1(a,b,cin,b_inv,select,result,cout);

input a;

input b;

input cin;

input b_inv;

input [1:0] select;

output result;

output cout;

wire bout;

wire add_result;

wire or_result;

wire and_result;

xor x1(bout,b_inv,b);

logic_and U4(a,bout,and_result);

logic_or U5(a,bout,or_result);

fulladder U3(a,bout,cin,add_result,cout);

mux4x1 U6(and_result,or_result,add_result,1'b0,select[1],select[0],result);

endmodule

module testbench();

reg a;
```

```
reg b;

reg cin;

reg b_inv;

reg [1:0] select;

wire result;

wire cout;

alubit1 unit (a,b,cin,b_inv,select,result,cout);

initial

begin

 //  AND

 a=0; b=0; cin=0; b_inv=0; select=2'b00;

#50

 a=0; b=1; cin=0; b_inv=0; select=2'b00;

#50

 a=1; b=0; cin=0; b_inv=0; select=2'b00;

#50

 a=1; b=1; cin=0; b_inv=0; select=2'b00;

#50

 //  OR

 a=0; b=0; cin=0; b_inv=0; select=2'b01;

#50

 a=0; b=1; cin=0; b_inv=0; select=2'b01;

#50
```

```
 a=1; b=0; cin=0; b_inv=0; select=2'b01;

#50

 a=1; b=1; cin=0; b_inv=0; select=2'b01;

#50

 //  ADD, cin=0

 a=0; b=0; cin=0; b_inv=0; select=2'b10;

#50

 a=0; b=1; cin=0; b_inv=0; select=2'b10;

#50

 a=1; b=0; cin=0; b_inv=0; select=2'b10;

#50

 a=1; b=1; cin=0; b_inv=0; select=2'b10;

#50

 //  ADD with cin=1

 a=0; b=0; cin=1; b_inv=0; select=2'b10;

#50

 a=0; b=1; cin=1; b_inv=0; select=2'b10;

#50

 a=1; b=0; cin=1; b_inv=0; select=2'b10;

#50

 a=1; b=1; cin=1; b_inv=0; select=2'b10;

#50

 //  SUB  select=10, b_inv=1, cin=1 => A + ~B + 1
```

```
 a=0; b=0; cin=1; b_inv=1; select=2'b10;

#50

 a=0; b=1; cin=1; b_inv=1; select=2'b10;

#50

 a=1; b=0; cin=1; b_inv=1; select=2'b10;

#50

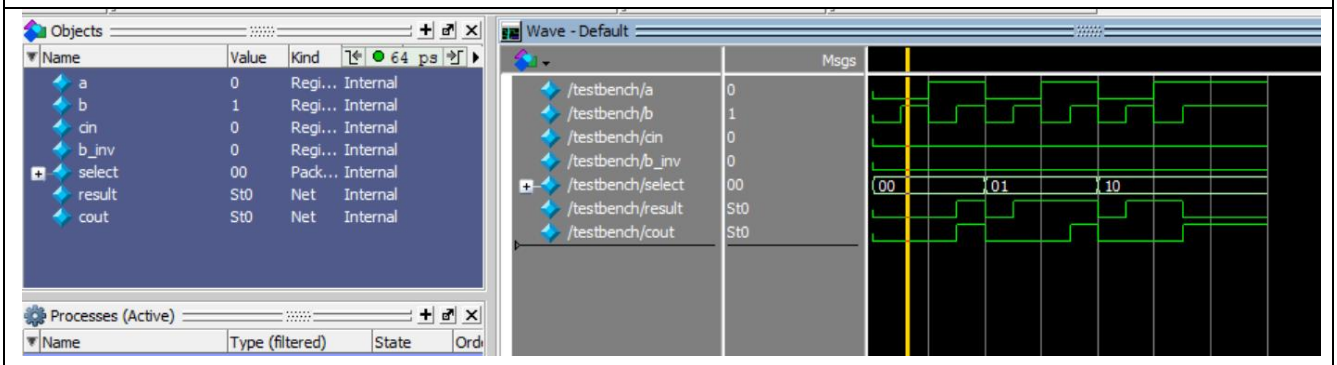
 a=1; b=1; cin=1; b_inv=1; select=2'b10;

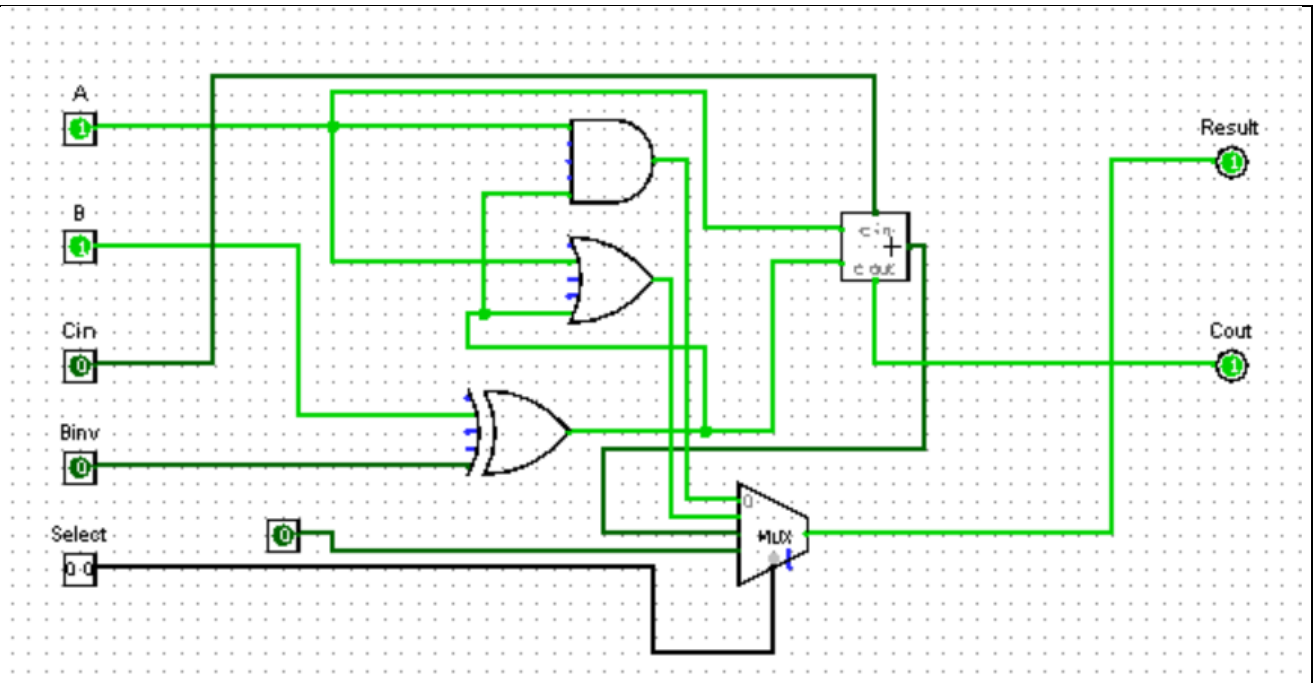
#50

end

endmodule
```

Output:





Task 2:

Description:
In this task we have to design 4 bit alu by using 1 bit alu and also we extend the operations SLT and result will be also in 4 bit .we will use flags CF,SF,ZF,OF that will tell the state of result like zero negative carry or overflow. This line assign signed_less = ($signed(a) < $signed(b)) will checks a is smaller than b or not according to signed comparison.
Code:

//halfadder
 	
module halfadder(a, b, s, c);
 	
input a;
 	
input b;
 	
output s;

```
output c;

xor x1(s, a, b);

and a1(c, a, b);

endmodule



//fulladder

module fulladder(a, b, cin, s, c);

input a;

input b;

input cin;

output s;

output c;

wire t1;

wire t2;

wire t3;

halfadder U1(a, b, t1, t2);

halfadder U2(t1, cin, s, t3);

or o1(c, t2, t3);

endmodule



//and

module logic_and(a, b, c);

input a;
```

```
input b;

output c;

and a1(c, a, b);

endmodule



//or

module logic_or(a, b, c);

input a;

input b;

output c;

or a1(c, a, b);

endmodule



//mux

module mux4x1(d0, d1, d2, d3, s1, s0, y);

input d0; //AND 00

input d1; //OR  01

input d2; //10 ADD/SUB

input d3; //11 SLT (or 0 for upper bits)

input s1;

input s0;

output y;

wire t1;
```

```
wire t0;

wire b0;

wire b1;

wire b2;

wire b3;

not n2(t1, s1);

not n3(t0, s0);

and a0(b0, d0, t1, t0);

and a1(b1, d1, t1, s0);

and a2(b2, d2, s1, t0);

and a3(b3, d3, s1, s0);

or o1(y, b0, b1, b2, b3);

endmodule



//1_bitALU

module alubit1(a, b, cin, b_inv, slt_in, select, result, cout);

input a;

input b;

input cin;

input b_inv;

input slt_in;

input [1:0] select;

output result;
```

```
output cout;

wire bout;

wire add_result;

wire or_result;

wire and_result;

xor x1(bout, b_inv, b);

logic_and U4(a, bout, and_result);

logic_or U5(a, bout, or_result);

fulladder U3(a, bout, cin, add_result, cout);

mux4x1 U6(and_result, or_result, add_result, slt_in, select[1], select[0], result);

endmodule



//4_bitALU

module alu4bit(a, b, binv, select, result, CF, SF, ZF, OF);

input [3:0] a;

input [3:0] b;

input binv;

input [1:0] select;

output [3:0] result;

output reg CF;

output SF;

output ZF;

output reg OF;
```

```
wire c0, c1, c2, c3;

wire signed_less;



assign signed_less = ($signed(a) < $signed(b));



alubit1 bit0(a[0], b[0], binv, binv, signed_less, select, result[0], c0);

alubit1 bit1(a[1], b[1], c0, binv, 1'b0, select, result[1], c1);

alubit1 bit2(a[2], b[2], c1, binv, 1'b0, select, result[2], c2);

alubit1 bit3(a[3], b[3], c2, binv, 1'b0, select, result[3], c3);



assign SF = result[3];

assign ZF = (result == 4'b0000);



// flags

always @(*) begin

   CF = 1'b0;

   OF = 1'b0;

   if(select == 2'b10) begin

      if(binv == 1'b0) begin

         CF = c3;

         OF = (~(a[3] ^ b[3])) & (result[3] ^ a[3]);
```

```
      end

      else begin

         CF = (a < b);

         OF = (a[3] ^ b[3]) & (result[3] ^ a[3]);

      end

   end

end

endmodule




module tb_alu4bit();

reg [3:0] a;

reg [3:0] b;

reg binv;

reg [1:0] select;

wire [3:0] result;

wire CF, SF, ZF, OF;



alu4bit uut (a, b, binv, select, result, CF, SF, ZF, OF);



initial

 begin

   //  AND (select = 2'b00)
```

```
    a = 4'b1010; b = 4'b1100; binv = 0; select = 2'b00;

    #50;



    // OR (select = 2'b01)

    a = 4'b1010; b = 4'b1100; binv = 0; select = 2'b01;

    #50;



    //  ADD (select = 2'b10, binv = 0)

    a = 4'b0101; b = 4'b0011; binv = 0; select = 2'b10;

    #50;



    //  SUB (select = 2'b10, binv = 1)

    a = 4'b0110; b = 4'b0010; binv = 1; select = 2'b10;

    #50;



    //  SLT (select = 2'b11) -> Checks if A < B signed

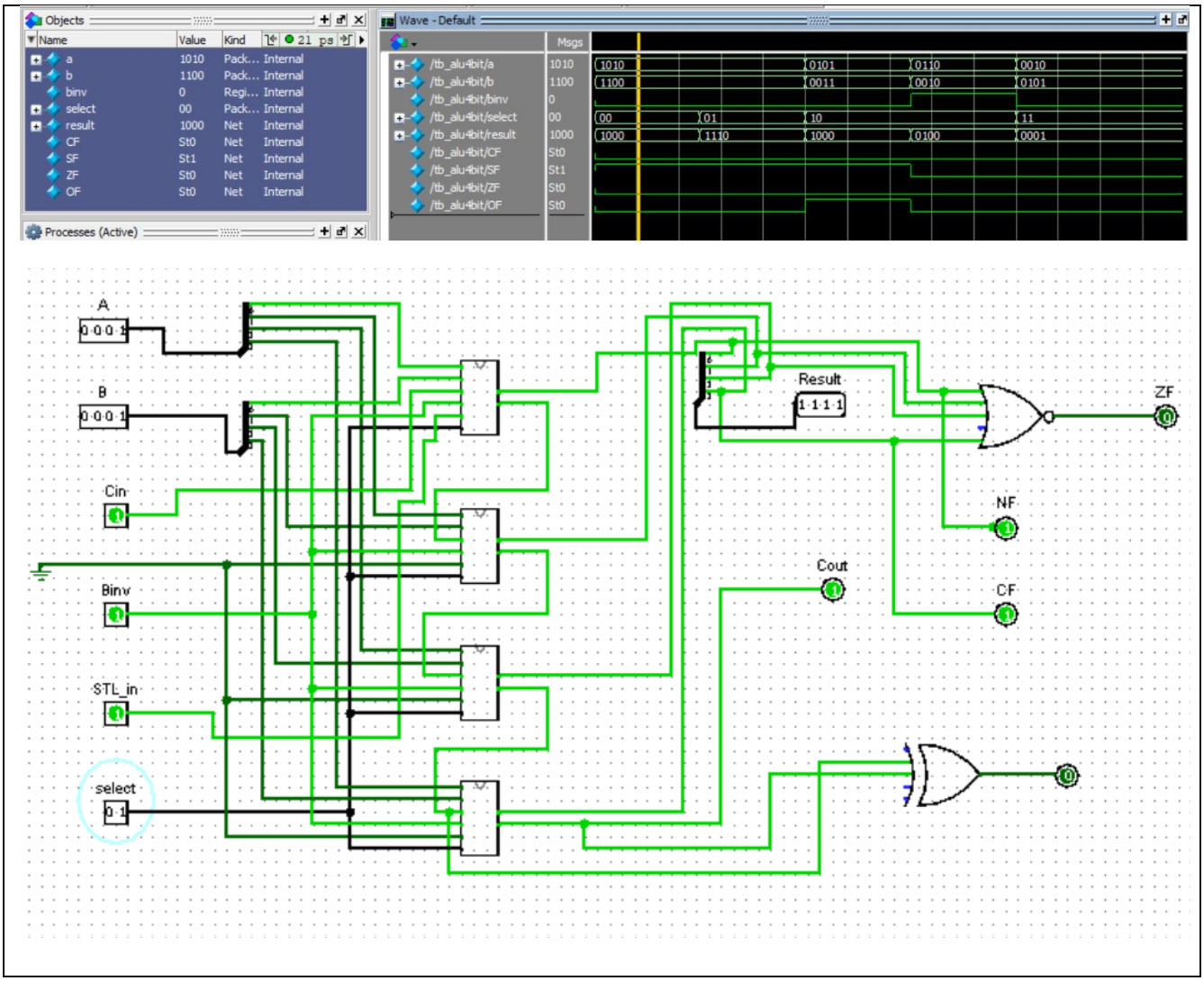
    a = 4'b0010; b = 4'b0101; binv = 0; select = 2'b11;

    #50;

end

endmodule
```

Output:



Task 3:

Description:
In this task we are required to make a paramterized alu that can be mad 4 but 8 bit etc, A control signl is used that decide which operation of ALU will be performed.here we kept N=8 means 8 bit alu.we take n bit numbers and operand and operations that will be performed by alu.then we define the combinational logic and case statement and used always block here then perform testbench.
Code:
module aluNbehav #(parameter N=8)(input [N-1:0] a, b, input [2:0] op,

```
    output reg [N-1:0] result

);

    always @(*) begin

        case(op)

            3'b000: result = a + b;

            3'b001: result = a - b;

            3'b010: result = a & b;

            3'b011: result = a | b;

            3'b100: result = ~a;

            3'b101: result = a ^ b;

            default: result = {N{1'b0}};

        endcase

    end

endmodule



module testbench();

    parameter N = 8;

    reg [N-1:0] a, b;

    reg [2:0] op;

    wire [N-1:0] result;

    integer i;

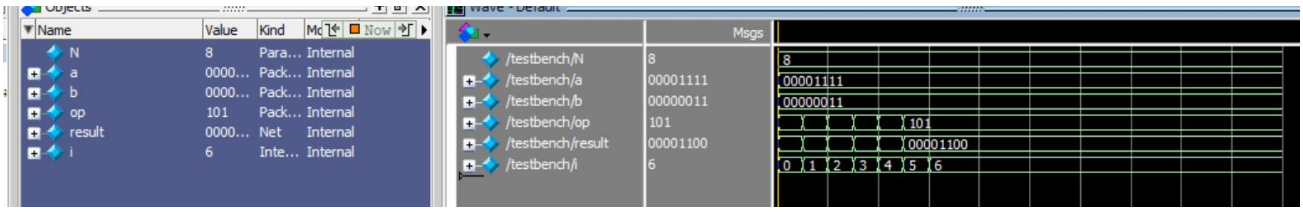


    aluNbehav #(.N(N)) uut (a, b, op, result);
```

```
  initial begin
    a = 0;
    b = 0;
    op = 0;

    for(i = 0; i < 6; i = i + 1) begin
      a = 15;
      b = 3;
      op = i;
      #10;
    end
  end
endmodule
```

Output:



Task 4:

Description:

Code:

```
module alu32bit (
   input  wire [31:0] a,
   input  wire [31:0] b,
   input  wire [3:0]  alu_op,
   output reg  [31:0] result,
   output reg         zero       // Zero flag
);

   always @(*) begin
      case (alu_op)
         4'b0000: result = a + b;                          // ADD
         4'b0001: result = a - b;                          // SUB
         4'b0010: result = a & b;                          // AND
         4'b0011: result = a | b;                          // OR
         4'b0100: result = a ^ b;                          // XOR
         4'b0101: result = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0; // SLT (Signed)
         4'b0110: result = (a < b) ? 32'd1 : 32'd0;              // SLTU (Unsigned)
         4'b0111: result = a << b[4:0];                    // SLL (Shift Left Logical)
         4'b1000: result = a >> b[4:0];                    // SRL (Shift Right Logical)
         4'b1001: result = $signed(a) >>> b[4:0];              // SRA (Shift Right Arithmetic)
         default: result = 32'd0;                          // Default case
```

```
        endcase



        zero = (result == 32'd0);

    end



endmodule




\

module testbench();



    reg  [31:0] a, b;

    reg  [3:0]  alu_op;

    wire [31:0] result;

    wire        zero;

    integer     i;



    alu32bit uut (

        .a(a),

        .b(b),

        .alu_op(alu_op),

        .result(result),

        .zero(zero)
```

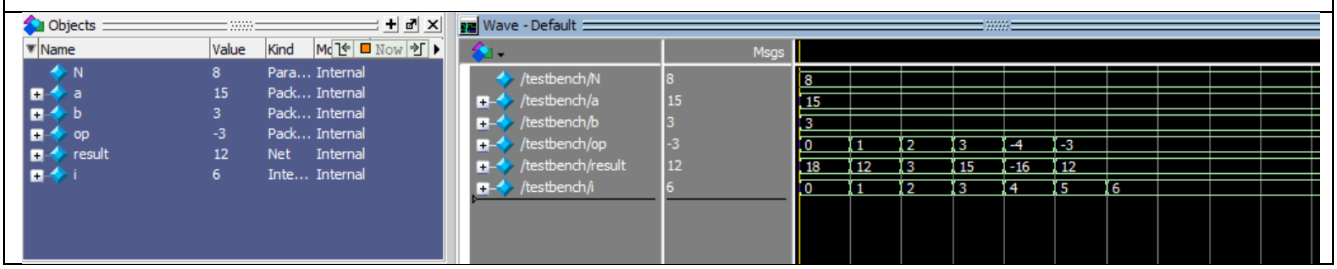
```
);

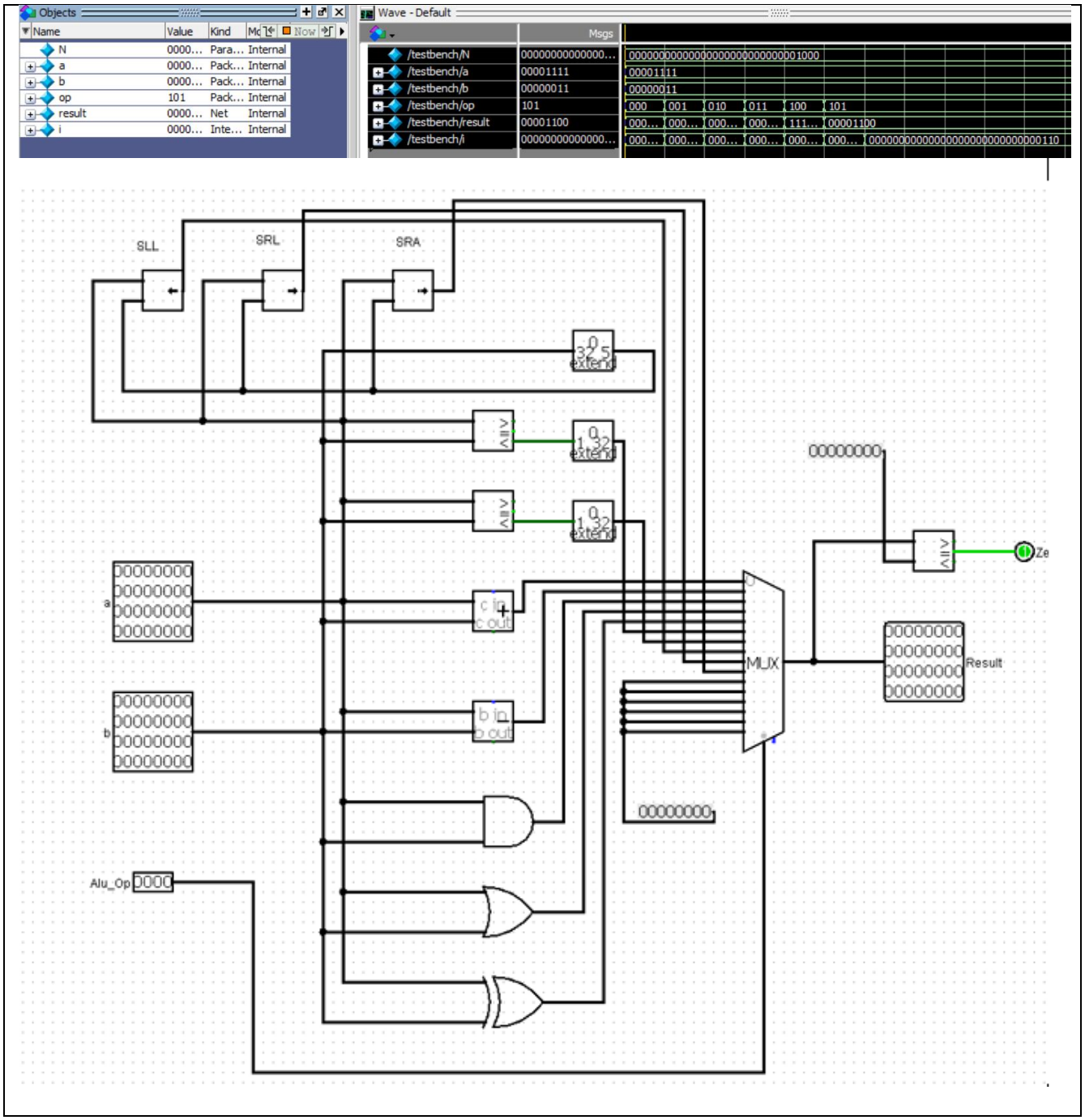

initial begin
    for (i = 0; i < 10; i = i + 1) begin
        alu_op = i[3:0];

        a = 32'h00000000; b = 32'h00000000; #50;
        a = 32'hFFFFFFFF; b = 32'h00000001; #50;
        a = 32'h80000000; b = 32'h00000001; #50;
        a = 32'h7FFFFFFF; b = 32'hFFFFFFFF; #50;
        a = 32'h12345678; b = 32'h12345678; #50;
        a = 32'hF0F0F0F0; b = 32'h0000001F; #50;
        a = 32'h80000000; b = 32'h00000004; #50;
        a = 32'hA5A5A5A5; b = 32'h0000000C; #50;
    end
end

endmodule
```

Output:




Objects
Name
Value
Kind
N
a
b
op
result
i
Wave - Default
Msgs
/testbench/N
/testbench/a
/testbench/b
/testbench/op
/testbench/result
/testbench/i
SLL
SRL
SRA
extend
MUX
Result
Alu_Op
Ze